

CV Platform

Comprehensive System Architecture & Component Documentation

A Detailed Design Guide for AI-Powered CV Extraction, Skill Normalization, and ATS Matching Scorer Engine

Version: 1.0.0 (Production Release)

Author: Lead AI Solutions Architect

Technology Stack: Next.js (V14), FastAPI, Supabase, SpaCy, Transformers, Ollama, Llama 3.1

Date: June 2026

1. Executive Summary & Table of Contents

The CV Platform is an enterprise-grade application designed to automate the lifecycle of talent profiling. It ingests unstructured resumes (PDF/DOCX), processes them through advanced layout-aware parsers, cleans the text, segments content into structured sections, runs Named Entity Recognition (NER), normalizes skills against the ESCO taxonomy, and computes an ATS Match Score against job descriptions using SHAP value explanations.

Document Outline & Table of Contents:

- Page 1: Cover Page
- Page 2: Table of Contents & Executive Summary
- Page 3: Core System Architecture & Directory Layout
- Page 4: Frontend App Directory & Layouts
- Page 5: Frontend Page Routers & Dynamic Components
- Page 6: Backend Base Setup & Utility Modules
- Page 7: Backend API Routers & Endpoints
- Page 8: Extraction & Natural Language Processing (NLP) Engine
- Page 9: Named Entity Recognition (NER) & Profile Parsing
- Page 10: Skill Normalization Engine & ESCO Taxonomy
- Page 11: ATS Match Scoring & SHAP Explainer
- Page 12: AI Improvement Engine & Local RAG Assistant
- Page 13: Database & Cloud Storage Architecture
- Page 14: Automated MLflow Tracking System
- Page 15: Containerization & DevOps Setup

2. Core System Architecture & Directory Layout

The CV Platform is designed as a decoupled, microservices-oriented architecture consisting of three primary layers:

- 1. **Frontend Layer:** A Next.js Web Application written in TypeScript, using TailwindCSS and custom Liquid Glass components.
- 2. **Backend Layer:** A high-performance FastAPI service running Python 3.11/3.13, orchestrating NLP pipelines and AI models.
- 3. **Database and Storage Layer:** Powered by Supabase, providing PostgreSQL, RLS policies, and raw file storage buckets.

Directory Structure Overview:

```
cv-platform/
├── backend/                # FastAPI Python Backend
│   ├── app/                # API Source Code
│   │   ├── routers/        # Endpoints definitions
│   │   ├── services/        # NLP, ESCO, ATS models
│   │   └── utils/           # Database, helpers
│   └── generate_perfect_cv.py # CV generation utility
├── frontend/               # Next.js Frontend Application
│   ├── app/                # Next.js routers & styles
│   ├── components/          # Translucent glass UI components
│   └── lib/                  # Store & API Axios clients
└── docker-compose.yml       # Multi-container local deployment
```

3. Frontend App Directory & Layouts

The frontend is structured around Next.js App Router. The styling system is configured using Vanilla TailwindCSS interleaved with custom glassmorphism styles defined in ``globals.css``.

Key Global Files:

1. `frontend/app/globals.css`:

Defines the visual custom properties for the Liquid Glass system. Variables like `--glass-bg`, `--glass-border`, and `--glass-shadow` are set to translucent whites with subtle contrast borders under a general off-white body background. It also includes keyframe animations like `fadeInUp`, `shimmer`, and `borderPulse` to provide micro-animations for card entries and loadings.

2. `frontend/app/layout.tsx`:

Sets the default HTML skeleton, imports Google Font *Inter* via Next.js Font Optimizations, and sets the layout background to the off-white CSS gradient. It wraps the entire document inside the `AppProvider` to store state variables across pages, and displays the global sticky Navbar.

3. `frontend/components/Navbar.tsx`:

A sticky, frosted glass navigation header. It dynamically highlights the current path and fetches parameters from the global store. For example, if a candidate has been analyzed, it appends `candidate_id` query parameters automatically to navigations like 'Score Match' and 'Improve CV' so users do not lose their current profile.

4. Frontend Page Routers & Dynamic Components

The client application is organized into 4 main page routers, each corresponding to a major user goal:

App Pages & Files:

1. `frontend/app/page.tsx` (Upload Page):

Responsible for handling drag-and-drop file inputs (PDF/DOCX) using `react-dropzone`. Once a file is drop-selected, it triggers the `/upload` API endpoint, updating upload progress dynamically. After raw text extraction completes, it renders an editable CV Parser Preview card allowing the candidate to fix fields in real-time. Changes are saved back to the database via a `Save Details` button before redirecting to scoring.

2. `frontend/app/score/page.tsx` (Match Scorer):

Computes the compliance between a CV and a job description. It renders a circular progress gauge representing the final ATS score, prints top positive and negative SHAP attribution cards, and formats matched/missing skills side-by-side. It also contains a timeline representation of suggested study paths.

3. `frontend/app/improve/page.tsx` (AI Optimizer):

An AI-powered 3-tab workbench to improve the resume. Includes: 1. *Bullet Points Optimizer* (rewriting experience points using metrics), 2. *Skill Gaps* (table of course/project curriculum suggestions), and 3. *Chat Assistant* (a RAG chat form).

4. `frontend/app/dashboard/page.tsx` (Analytics):

Collects and charts platform-wide aggregate metrics (total candidates, total matches, average scores) and displays a catalog list of previously processed candidate records.

5. Backend Base Setup & Utility Modules

The backend is powered by FastAPI. Uvicorn acts as the ASGI web server, supporting live-reloading during local development. The system coordinates configurations using environment variables mounted from ``.env`` files.

Core Backend Modules:

1. `backend/app/main.py`:

The application bootstrapper. It initializes the FastAPI instance and mounts the CORS middleware, explicitly authorizing cross-origin resource sharing from the local frontend origin (`http://localhost:3000`). It also aggregates and mounts routers for all modules under the unified `/api/v1` prefix.

2. `backend/app/utis/supabase_client.py`:

Serves as the database and storage driver. It wraps the official Supabase Python client using environment keys (`SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY`). It exposes wrapper functions for:

- `upload_file_to_storage`: Uploads raw PDF/DOCX bytes to the resumes bucket.
- `insert_candidate` / `update_candidate`: Updates parsing details and state fields in the database.
- `bulk_insert_extracted_skills`: Stores NLP-mapped skills into the `extracted_skills` relation table.

6. Backend API Routers & Endpoints

The backend divides routing logic into distinct sub-modules inside the `app/routers/` directory, mapping RESTful routes to specific database and NLP services:

Active API Endpoints:

1. `upload.py` -> **POST** `/api/v1/upload`:

Handles multipart form file uploads. Validates file types (PDF/DOCX) and file sizes (< 10MB). Triggers the extraction engine, writes the raw document bytes to Supabase Storage, registers the candidate in the database with status extracted, and returns the parsed character count and raw text payload.

2. `parse.py` -> **POST** `/api/v1/parse/{candidate_id}` & **PUT** `/api/v1/candidates/{candidate_id}`:

The POST route runs text cleaning, section detection, hybrid Named Entity Recognition, and ESCO skill normalization. It updates the candidate row status to parsed. The PUT route enables frontend manual edits to be stored.

3. `jobs.py` -> **POST** `/api/v1/jobs`:

Registers a target job description and extracts its key tech keywords and role parameters.

4. `score.py` -> **POST** `/api/v1/score`:

Orchestrates the ATS scoring matching logic and computes SHAP value attributions.

5. `ai.py` & `dashboard.py` -> **GET** `/api/v1/dashboard`:

`ai.py` mounts rewriter, gaps, and chat prompts. `dashboard.py` provides metric counts and logs.

7. Extraction & Text NLP Engine

Before applying deep learning models, raw documents must be converted into clean, contiguous text. This is handled by dedicated text processing services in the `app/services/` directory:

Files & Algorithms:

1. `app/services/extractor.py` (Document Reader):

Utilizes PyMuPDF (imported as `fitz`) to read PDFs. It extracts text with layout-aware parameters (`sort=True`) ensuring multi-column text blocks are grouped vertically instead of collapsing horizontally. For Word documents, it parses paragraphs and tables using the `python-docx` library.

2. `app/services/cleaner.py` (Text Normalizer):

Cleans the raw text. It decodes smart quotes (e.g. converting curly quotes to straight quotes), maps em dashes, strips non-printable control symbols, normalizes unicode spacing, and collapses redundant line breaks. This prevents formatting anomalies from disrupting NLP classifiers.

3. `app/services/section_detector.py` (Section Segmentation):

Scans the cleaned text line-by-line to detect section boundaries. It uses case-insensitive regular expressions matching common headings (e.g. Summary, Education, Experience, Skills, Projects, Certifications, Languages, Achievements). Once boundaries are established, it returns segmented text chunks. Chunks with less than 20 characters are flagged as low confidence, while larger blocks are labeled high confidence.

8. Named Entity Recognition (NER) & Profile Parsing

Extracting contact info (name, email, phone, location) and professional duration from unstructured text is achieved via a hybrid Named Entity Recognition pipeline inside `ner_orchestrator.py`.

Hybrid Pipeline Steps:

Step 1: SpaCy NER Classifier:

The system loads the spaCy model `en_core_web_sm` to extract standard entities like PERSON, EMAIL, GPE (locations), and ORG (companies/universities) from the personal info section.

Step 2: BERT fallback:

If spaCy fails to isolate a name or email, it calls the HuggingFace transformer pipeline `dslim/bert-base-NER`. The system merges entities, resolving duplicate token spans.

Step 3: Location GPE Blacklisting:

Common technical keywords (like AI, UI, Next.js) are frequently misclassified by standard models as location entities (GPE). The GPE parser filters extracted strings against a strict blacklist to ensure locations like 'Karachi, Pakistan' are correctly preserved.

Step 4: Experience Duration Summing:

The system scans the experience text for date ranges (e.g., 'Aug 2022 - May 2024') using regex. It parses month and year boundaries using `python-dateutil`, sums the durations, and returns the total experience in years.

9. Skill Normalization Engine & ESCO Taxonomy

To make resume skills searchable and comparable, they must be normalized against a standardized taxonomy. The CV Platform utilizes the European Skills, Competences, Qualifications and Occupations (ESCO) taxonomy.

Normalization Mechanics:

1. Token splitting:

The skills section text is tokenized using parenthesis-aware delimiters. It splits on commas, semicolons, slashes, and pipes (e.g. `,`, `/`, `|`) only when they reside outside of parenthesis. This ensures that compound skill descriptions like 'React Native (Core / Bridge)' are preserved as a single item instead of being split incorrectly.

2. Embedding search:

The normalizer loads the SentenceTransformer model `all-MiniLM-L6-v2`. It generates vector embeddings for the extracted skill string and performs cosine similarity search against a pre-compiled dataset of ESCO preferred skill terms. On Apple Silicon, it accelerates calculations using the PyTorch mps (Metal) GPU backend.

3. Tech Overrides & RapidFuzz Fallback:

For common software terms (e.g. `FastAPI`, `Vite`) that might not map cleanly in ESCO, a hardcoded dictionary overrides standard searches. If similarity scores fall below 0.7, the engine runs a string edit distance search using the `RapidFuzz` library as a fallback to resolve typos.

10. ATS Match Scoring & SHAP Explainer

The ATS Match Scorer (app/services/ats_scorer.py) calculates the overall match score between a candidate profile and a job description. The score is computed as a weighted average across multiple criteria:

Weighted Scoring Rubric:

- **Skills Alignment (40%)**: Computes the ratio of matched normalized skills to target job requirements.
- **Experience Match (20%)**: Audits total candidate experience years against target job requirements.
- **Semantic Similarity (15%)**: Runs sentence embedding similarity between the resume text and the job description.
- **Education Audit (10%)**: Scans the education block for matching degree titles (e.g., Bachelor, Master, PhD).
- **Project Relevance (5%)**: Compares project details to job criteria using word embeddings.
- **Certifications (5%)**: Checks for presence of industry certifications.
- **Resume Quality (5%)**: Audits text parameters like spelling and section structure.

SHAP Feature Contributions:

Rather than outputting a black-box percentage, the scorer calculates **SHAP (SHapley Additive exPlanations)** values for each category. It models the scoring equation as a game where each sub-criteria is a player. It outputs top positive contributions (e.g., '+15% due to Python skill match') and top negative contributions (e.g., '-10% due to missing cloud deployment experience') to help candidates audit their score.

11. AI Improvement Engine & Local RAG Assistant

Once gaps are identified, the CV Platform provides automated suggestions via local AI endpoints, prompting an Ollama server running Llama 3.1 8B Instruct model:

Key Improvement Features:

1. Bullet Points Optimizer:

Accepts a target role and original resume bullets. The Llama 3.1 model refactors each bullet point, replacing weak verbs with strong action verbs (e.g. 'wrote Python code' -> 'Engineered Python API layers') and inserts metric placeholders (e.g. '[X]%)' to encourage quantitative impact descriptions.

2. Skill Gap curriculum builder:

For each missing skill identified in the scoring phase, the system prompts Llama 3.1 to return a study roadmap. The model suggests: 1. A free online learning course name, 2. A hands-on practice project to build, and 3. An estimated study period in weeks, representing priority levels (High/Medium/Low) for acquisition.

3. RAG Chat Assistant:

Supports direct career Q&A.; The candidate's resume and current ATS score audit details are loaded into a vector context store. When the user asks a question (e.g. 'Why is my ATS score low?'), the system retrieves matching context blocks, injecting them into the system prompt of Llama 3.1 for a context-aware answer.

12. Database & Cloud Storage Architecture

The database layer is managed by PostgreSQL hosted on Supabase. Relational models enforce integrity across candidates, skills, and evaluation scores.

Core Database Tables:

1. candidates table:

Stores the primary profile records. Columns include: id (UUID primary key), file_name, file_path (Supabase Storage reference path), candidate_name, email, phone, location, linkedin_url, total_experience_years, raw_text, raw_sections (JSONB storing split sections), raw_entities (JSONB storing entities), status (extracted/parsed), and created_at.

2. extracted_skills table:

Stores the NLP-normalized skills. Columns include: id, candidate_id (Foreign Key referencing candidates), skill_raw, skill_normalized (matched ESCO label), confidence, and created_at.

3. ats_scores table:

Logs matching history. Columns include: id, candidate_id, job_title, ats_score (numeric percentage), grade, breakdown (JSONB storing category scores), recommendation, created_at.

Cloud Storage Buckets:

The platform includes a storage bucket named resumes. Row-Level Security (RLS) public policies allow write (INSERT) and read (SELECT) access to raw resume PDF files.

13. Automated MLflow Tracking System

To monitor parsing quality and model drift, the platform integrates **MLflow Tracking**. Whenever a CV is analyzed, a tracking run is initiated.

MLflow Logs Architecture:

1. Run Context:

Runs are registered under a default experiment name. MLflow stores tracking data locally in `./ml/mlruns`, ensuring offline functionality.

2. Logged Parameters:

- `candidate_id`: Unique reference key.
- `ner_method`: Method utilized (e.g. `spacy` or `bert fallback`).
- `skill_count`: Total skills extracted.
- `char_count`: Size of raw extracted text.

3. Logged Metrics:

- `parsing_duration_seconds`: Time taken by the pipeline to segment and tag.
- `total_experience_years`: Numeric experience years.

4. Logged Artifacts:

- `raw_text.txt`: The complete cleaned raw text parsed from PyMuPDF.
- `parsed_profile.json`: The structured candidate payload returned by the NER engine.

14. Containerization & DevOps Setup

The entire platform is containerized using Docker to ensure consistent local development and production environments.

Service Specifications:

1. backend (FastAPI):

Uses python:3.11-slim. Installs compiler tools, cmake (required for FAISS compiling), and graphics tools (needed for PyMuPDF layout and OCR fallbacks). Exposes port 8000.

2. frontend (Next.js):

Multi-stage build using node:18-alpine. Compiles static production pages, exposing port 3000.

3. ollama (LLM Server):

Runs the official ollama/ollama:latest image. Mounts a volume named ollama_models to cache downloaded LLM weights, exposing port 11434.

4. ollama-pull (Companion Helper):

A slim service that automatically triggers a curl request to Ollama once the container boots, initiating the download of the llama3.1:8b-instruct-q4_K_M weights.

5. mlflow (Tracking Server):

Mounts ./ml/mlruns to store metadata. Exposes port 5000 inside the network, mapped to host port 5050 to avoid AirPlay conflicts on macOS.